Portfolio Crisis Twin

A flight simulator for your money

Multi-Agent Autonomous Portfolio Management and Risk Intelligence Platform

Feature selection, implementation plan, architecture, validation strategy, demo script and internship-readiness guide

Item	Detail
Target	Meta developer AI hackathon (Muse or other model APIs); reusable for any hackathon and fintech internship applications
Mode	Simulation and paper trading only. No real money, not financial advice.
Core MVP	Portfolio Crisis Twin: stress simulator with Bull, Bear and Risk Auditor agents
Mandatory feature	Voice: Voice Portfolio War Room and Crisis Drill mode
Team	Manan Soni, Aditya Kumar Singh, Shivansh Gupta (Mentor: Piyush Gupta)

1. Positioning: why this is not a regular portfolio tool

Regular portfolio tools answer **"what should I buy?"**. Crisis Twin answers **"what could break me, and would I survive it?"** Everything in the product serves one pitch line: *pilots train in simulators, investors train with real money; we built the simulator.*

Design principles

- **One sharp identity.** Do not pitch a list of agents. Pitch a simulator with a debate and a voice.
- **Numbers come from code, reasoning comes from the LLM.** The language model translates scenarios and explains results; a quantitative engine computes impact.
- **Show disagreement.** Preserve dissent between agents instead of hiding it behind one confident answer.
- **Be honest about uncertainty.** Show ranges and confidence flags, not single false-precision numbers.
- **Validate on real history.** Replay 2008, COVID-2020 and 2022 to prove the simulator is credible.

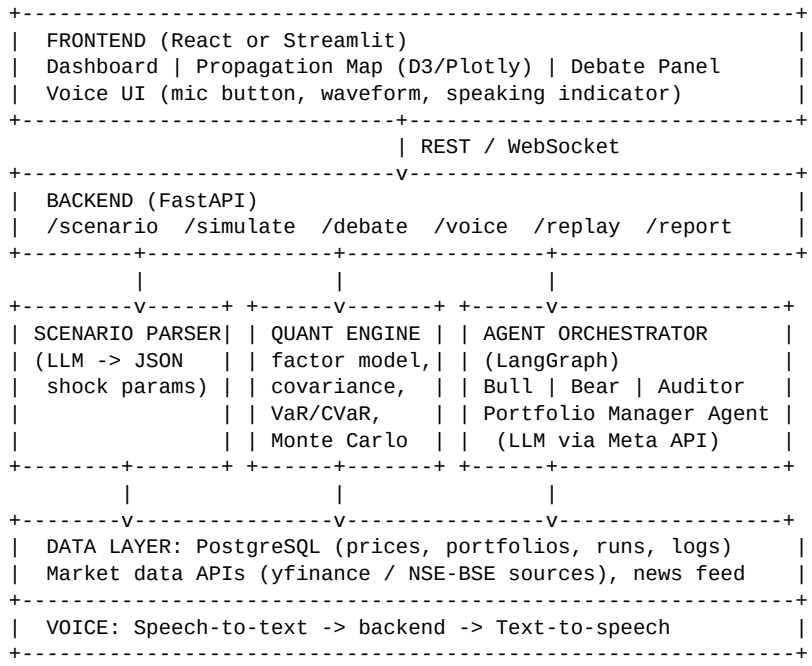
2. Final feature set at a glance

#	Feature	What the user sees	Priority	Effort
1	Portfolio Crisis Twin	Type or speak a crisis; shock is applied to the portfolio; before/after comparison	Must	High
2	Bull / Bear / Risk Auditor debate	Three agents argue; dissent and scoreboard are visible	Must	Medium
3	Voice War Room	Ask and command by voice; agents answer in distinct voices; interrupt allowed	Must (mandatory)	Medium
4	Crisis Drill mode	Newsroom-style timed alert; user decides by voice; behavior compared to stated risk appetite	Should	Medium
5	Shock Propagation Map	Animated graph: shock spreads across sectors to holdings; hidden exposures light up	Must (wow factor)	Medium
6	Regret Replay	What if you had followed the suggestions in a real past crisis	Should (credibility)	Medium
7	Crisis Report Card	Shareable PDF/image with survival score, hidden risk, fix	Should	Low
8	Uncertainty and audit trail	Ranges, confidence flags, full log of every recommendation	Should	Low
9	Why Did My Portfolio Move?	Plain-language attribution of a day's move	Optional	Medium
10	Behavioral risk profile	Risk tolerance from scenarios, not a form	Optional / cut first	Low

Cut or de-emphasize: long lists of agents (technical, sentiment and forecasting can exist but should not be the demo), price prediction (most overclaimed feature in this genre) and generic dashboards.

3. System architecture

A layered design keeps the LLM away from the arithmetic and makes every result traceable.



Technology stack

Layer	Choice	Notes
LLM	Meta model API (Muse or other model offered by the hackathon)	Check the hackathon docs for exact endpoints, limits and function-calling support. Wrap calls in one client class so the model can be swapped.
Agents	LangGraph (or LangChain)	Graph of nodes: parse -> simulate -> debate -> audit -> synthesize
Backend	FastAPI + Pydantic	Typed schemas for shock parameters and agent outputs
Quant	Python, NumPy, Pandas, SciPy, scikit-learn, statsmodels	Covariance (Ledoit-Wolf), regression betas, Monte Carlo
Database	PostgreSQL (SQLite for the hackathon demo)	Tables: assets, prices, portfolios, scenarios, runs, agent_logs
Frontend	React with D3 or Plotly, or Streamlit for speed	Propagation graph is the key visual
Voice	Web Speech API or Whisper for STT; browser TTS or a neural TTS service for TTS	Use different voice settings per agent
DevOps	Docker, GitHub Actions, pytest	Reproducible setup in under five minutes

4. Feature implementation guide

4.1 Portfolio Crisis Twin (core engine)

Goal: turn a crisis described in words into a quantified, explainable impact on the user's holdings.

Step-by-step

- **Parse the scenario.** The LLM converts text such as "oil +25% and rates +100 bps" into a structured shock object. It does not output impact numbers.
- **Map shocks to risk factors.** Factors: oil, interest rate, USD/INR, equity market, inflation proxy, volatility index, plus sector factors.
- **Estimate sensitivities (betas).** Regress each asset's returns on the factors over a rolling window (for example 3 to 5 years of daily or weekly returns).
- **Apply the shock.** Expected asset impact = sum of (beta to factor x factor shock). Add a stress-correlation adjustment because correlations rise in crises.
- **Quantify uncertainty.** Run Monte Carlo around the shock using the residual covariance to produce a range (for example 5th to 95th percentile loss).
- **Compute portfolio outputs.** Portfolio loss, loss by holding, VaR/CVaR before and after, risk contribution per asset.
- **Return a result object** that the agents, map and voice layer all consume.

Example shock schema

```
{
  "scenario_name": "Oil spike plus rate hike",
  "shocks": [
    {"factor": "oil_price",     "change_pct": 25.0, "confidence": 0.8},
    {"factor": "policy_rate",   "change_bps": 100,  "confidence": 0.7}
  ],
  "horizon_days": 10,
  "regime": "stress"
}
```

Core impact computation (simplified)

```
import numpy as np

def apply_shock(betas, shock_vec, weights, resid_cov, n_sims=5000, seed=42):
    # betas: (n_assets x n_factors), shock_vec: (n_factors,), weights: (n_assets,)
    rng = np.random.default_rng(seed)
    expected = betas @ shock_vec                        # per-asset expected return
    noise = rng.multivariate_normal(np.zeros(len(weights)), resid_cov, n_sims)
    sims = expected + noise                             # (n_sims x n_assets)
    port = sims @ weights
    return {
        "expected_loss": float(expected @ weights),
        "p05": float(np.percentile(port, 5)),
        "p95": float(np.percentile(port, 95)),
        "cvar95": float(port[port <= np.percentile(port, 5)].mean()),
        "per_asset": (expected * weights).tolist(),
    }
```

Common pitfalls: using the LLM to guess percentages; ignoring that betas change in crises; too short a data window; not flagging when a scenario is outside historical range.

4.2 Bull, Bear and Risk Auditor debate

Goal: adversarial reasoning over the quantitative results so the user sees both opportunity and danger.

Agent	Role	Inputs	Output
Bull Agent	Finds opportunities and defensible positions during the crisis	Simulation result, sector impacts, valuations	Proposed adjustments with rationale
Bear Agent	Attacks the proposal; finds ways it fails	Same, plus Bull's proposal	Counter-arguments, downside cases
Risk Auditor	Independent; checks hidden exposures and rule violations	Simulation result, correlation matrix, concentration stats	Hidden exposure list, limit breaches, verdict
Portfolio Manager	Synthesizes the debate into final recommendation	All of the above	Ranked actions, trade-offs, confidence

- **Orchestration.** LangGraph nodes: Bull -> Bear (sees Bull) -> Auditor (sees both and raw numbers) -> Portfolio Manager. Allow one rebuttal round to keep latency low.
- **Ground every claim.** Each agent receives the numeric result as JSON and must cite which number it relies on. Reject outputs whose cited numbers do not match.
- **Auditor uses code tools.** Give it functions such as `top_risk_contributors()` and `correlation_in_stress()` so it discovers exposures by computation, not guesswork.
- **Structured output.** Ask each agent for JSON (claim, evidence, confidence, suggested_action) and render it in the UI and voice layer.
- **Agent scoreboard.** Backtest each agent's past calls on historical crises and show hit rates. Weight the Portfolio Manager's synthesis by this record.
- **Preserve dissent.** If the Auditor disagrees with the final recommendation, display that explicitly.

Agent prompt skeleton (Bear Agent)

```
SYSTEM: You are the Bear Agent. Your job is to find the strongest reasons the
proposed strategy could fail. Use ONLY the numbers in SIM_RESULT. Cite the field
you rely on. Do not invent figures. Output JSON:
{"claims":[{"claim":str,"evidence_field":str,"severity":1-5}],
 "worst_case_story":str,"confidence":0-1}
USER: SIM_RESULT=<json> PROPOSAL=<bull json> PORTFOLIO=<json>
```

4.3 Voice Portfolio War Room (mandatory voice feature)

Goal: make voice structural to the experience, not a chatbot add-on.

Capabilities

- **Ask by voice:** "What happens if oil jumps 20%?" runs the Crisis Twin and speaks a summary.
- **Live debate:** Bull, Bear and Auditor speak in distinct voices. The user can interrupt: "Bear, respond to that."
- **Commands:** "Rebalance to lower risk and show before and after." "Add ten shares of HDFC Bank in paper trading."
- **Daily briefing:** a 60-second spoken summary of moves, risk changes and relevant news.

Voice pipeline

```
Mic -> Speech-to-text -> Intent router (LLM with fixed intent list) -> Action
   -> Backend endpoint (simulate / debate / trade / explain)
   -> Response text per agent -> Text-to-speech (voice per agent) -> Speaker
   -> Same text and visuals pushed to UI over WebSocket
```

Implementation steps

- **Capture audio** in the browser (MediaRecorder or Web Speech API). Show a live waveform so judges see it working.
- **Transcribe.** Web Speech API for zero setup, or a Whisper model for better accuracy on finance terms and Indian company names.
- **Route the intent.** Use the LLM with a fixed list: RUN_SCENARIO, START_DEBATE, INTERRUPT_AGENT, REBALANCE, PAPER_TRADE, EXPLAIN_MOVE, BRIEFING. Return JSON with parameters. Validate before executing; require spoken confirmation for any paper trade.
- **Speak with distinct voices.** Assign each agent a voice and speaking rate. Queue utterances so agents do not overlap.
- **Support interruption (barge-in).** Stop TTS playback when speech is detected, then route the new utterance with the debate context.
- **Keep it short.** Spoken answers should be 2 to 4 sentences with the detail shown on screen.
- **Fallback.** Always provide a text input and a typed transcript in case audio fails during the demo.

Intent router output example

```
User: "What if oil jumps twenty five percent and the Bear thinks my banks are safe?"
-> {"intent":"RUN_SCENARIO","scenario_text":"oil +25%","post_action":"START_DEBATE",
    "focus_agent":"bear"}
```

4.4 Crisis Drill mode (voice plus behavior)

Goal: create emotional engagement and reveal real behavior under pressure.

- A newsroom-style spoken alert announces the crisis: "Breaking: oil up 25% overnight. Your portfolio is down 6% pre-market. You have 60 seconds."
- A visible countdown runs while the user answers by voice (hold, sell X, hedge, buy defensive).
- The simulation applies the user's decision and the agents react live.
- Afterwards the system compares stated risk tolerance with drill behavior: "You said low-risk, but you sold at the bottom." Store results for the behavioral profile.
- **Implementation:** a state machine (ALERT, COUNTDOWN, DECISION, OUTCOME, DEBRIEF), a scenario library of three to five pre-built crises, and the same intent router as the War Room.

4.5 Shock Propagation Map (signature visual)

Goal: a 10-second visual that shows how a shock travels to the user's holdings, including indirect paths.

- **Graph construction.** Nodes: macro factors, sectors, companies. Edges: weights from factor betas, sector correlations and supply-chain or revenue-exposure links (a curated CSV of 30 to 60 links is enough for the demo).
- **Propagation algorithm.** Start at the shocked node, push impact along edges with decay per hop, accumulate at holdings. Record each path for explanation.
- **Hidden exposure detection.** Flag holdings that look diversified by sector but share indirect paths to the same shock.
- **Rendering.** D3 force graph or Plotly network. Animate by hop so nodes turn red in sequence; edge thickness shows transmission strength; clicking a holding lists its top paths.
- **Honesty.** Mark curated edges as assumptions and statistical edges as estimated; show confidence on edges.

```
def propagate(graph, shock_node, magnitude, decay=0.6, max_hops=4):
    impact = {shock_node: magnitude}; frontier = [(shock_node, magnitude, [shock_node])]
    paths = []
    for hop in range(max_hops):
        nxt = []
        for node, val, path in frontier:
            for nbr, w in graph[node]:
                v = val * w * decay
                if abs(v) > 1e-4:
                    impact[nbr] = impact.get(nbr, 0) + v
                    nxt.append((nbr, v, path + [nbr])); paths.append((path + [nbr], v))
        frontier = nxt
    return impact, paths
```

4.6 Regret Replay (credibility feature)

Goal: prove the simulator on real history and make the benefit personal.

- **Pick crises:** 2008 financial crisis, COVID crash (Feb to Mar 2020), 2022 rate-hike drawdown, plus an oil spike episode.
- **Out-of-sample calibration.** Fit betas only on data before the crisis, simulate the known shock, and compare predicted loss with the actual loss. Show this chart in the demo and README.
- **Counterfactual.** Apply the system's recommended adjustment at the crisis start (with transaction costs) and compare drawdown: "You would have lost 14% instead of 31%."
- **Report honestly.** Include the cases where predictions were off and explain why.
- **Avoid look-ahead bias:** use only information available on the decision date, including constituent lists where possible.

4.7 Crisis Report Card (shareable output)

- Generate a one-page PDF or image per simulation: scenario, survival score, loss range, biggest hidden exposure, recommended fix, before/after chart, disclaimer.
- **Implementation:** ReportLab or an HTML-to-PDF template filled from the result object; expose via `/report/{run_id}`; add a download button and QR link.

4.8 Uncertainty and audit trail

- Show ranges (for example "likely loss 8% to 14%") and a confidence flag: low when the scenario exceeds historical ranges or data coverage is thin.
- Log every run: inputs, parsed shocks, factor betas version, agent prompts and outputs, final recommendation, timestamp. Store in PostgreSQL `runs` and `agent_logs` tables.
- Add an "Explain this number" panel that lists the formula inputs behind any displayed figure.
- Include a visible disclaimer: simulation only, not financial advice, known model limitations listed.

4.9 Optional: Why Did My Portfolio Move?

Attribute a day or event move into contributors (holdings, sectors, factors, news) with a waterfall chart. Use factor decomposition for the numbers and SHAP-style attribution if a model is involved; let the LLM write a plain-language story and classify the move as noise, temporary shock or structural risk.

4.10 Optional: Behavioral risk profile

Collect four to five scenario choices (ideally from Crisis Drill) and compute loss tolerance, reaction style and horizon. Flag proposals whose worst case exceeds the user's likely breaking point. Cut this first if time is short.

5. Data, schema and APIs

Data sources

- **Prices:** yfinance or an exchange-data source for NSE/BSE and global tickers; daily data is sufficient. Cache locally.
- **Macro factors:** crude oil futures, USD/INR, government bond yields, India VIX, Nifty 50 index.
- **News:** RSS or news API for the sentiment context (keep optional).
- **Sector links:** a small hand-curated CSV for supply-chain and exposure edges.
- **Check licensing** of any data source before publishing the project publicly.

Core database tables

Table	Key columns
assets	id, ticker, name, sector, exchange
prices	asset_id, date, close, volume
factors	factor_id, name, date, value
portfolios / holdings	portfolio_id, asset_id, quantity, cost_basis
scenarios	id, text, parsed_shocks_json, created_at
runs	id, portfolio_id, scenario_id, result_json, model_version, created_at
agent_logs	run_id, agent, prompt, output_json, latency_ms
paper_trades	id, portfolio_id, asset_id, side, qty, price, ts

API endpoints

Endpoint	Purpose
POST /scenario/parse	Text to structured shock object (LLM)
POST /simulate	Run quant engine; returns impact, ranges, risk metrics, propagation paths
POST /debate	Run Bull, Bear, Auditor, Portfolio Manager; returns structured transcript
POST /voice/intent	Transcript to validated intent and parameters
POST /rebalance/preview	Before/after comparison for suggested adjustments
POST /paper/trade	Record simulated trade (requires confirmation flag)
GET /replay/{crisis}	Regret Replay results
GET /report/{run_id}	Crisis Report Card PDF

6. Quantitative rigor and evaluation

This section is what separates a demo from an internship-worthy project.

- **Risk metrics:** historical, parametric and Monte Carlo VaR; CVaR (Expected Shortfall); max drawdown; Sharpe, Sortino and Calmar ratios; risk contribution by asset; stress-regime correlation.
- **Backtesting:** walk-forward, transaction costs and slippage included, no look-ahead or survivorship bias.
- **Baselines:** equal-weight, 60/40, mean-variance, risk parity, buy-and-hold Nifty 50 or S&P 500.

- **Ablation studies:** with versus without the Bear/Auditor debate; with versus without the stress-correlation adjustment; LLM-guessed shocks versus calibrated shocks.
- **Scenario calibration:** predicted versus actual loss for at least three historical crises.
- **Agent evaluation:** does the Auditor catch exposures a plain covariance check misses? Does the Bear's critique reduce drawdown in replay?
- **Testing:** unit tests for portfolio math, VaR and propagation; integration test for the full simulate-and-debate flow; fixed random seeds.

7. Build plan

Adjust the number of days to the hackathon deadline. The order matters more than the duration: get an end-to-end thin slice working first, then deepen.

Phase	Deliverables	Done when
1. Thin slice	Data loader; sample portfolio; hard-coded shock; impact numbers on screen; LLM client wrapper	A typed scenario returns a before/after number end to end
2. Quant core	Factor betas, Monte Carlo ranges, VaR/CVaR, risk contributions	Unit tests pass; ranges look sensible
3. Agents	Scenario parser, Bull, Bear, Auditor, Portfolio Manager with structured JSON	Debate returns grounded, parseable output
4. Propagation map	Graph data, propagation function, animated visual	Shock visibly spreads and flags hidden exposure
5. Voice	STT, intent router, per-agent TTS, interruption, text fallback	Spoken scenario to spoken debate works reliably
6. Validation	Regret Replay on three crises, baseline comparison, calibration chart	Charts and honest numbers ready for demo and README
7. Crisis Drill and report card	Timed drill flow; PDF report	Drill runs in under two minutes
8. Polish	Docker, README, architecture diagram, demo video, disclaimer, rehearsal	Fresh clone runs in five minutes; demo rehearsed three times

Suggested team split

- **Member 1:** agents, orchestration, LLM integration, voice intent routing.
- **Member 2:** quant engine, data pipeline, backtesting and Regret Replay.
- **Member 3:** frontend, propagation map, voice UI, report card, demo polish.
- Everyone: shared JSON schemas defined on day one so work can proceed in parallel.

8. Three-minute demo script

Time	Action	What judges should feel
0:00	Hook: "Pilots train in simulators. Investors train with real money. We built the simulator."	A clear idea
0:15	Load a sample portfolio	Concrete and relatable
0:35	Speak: "What if oil jumps 25% and rates rise?"	Voice is real and natural
1:05	Propagation map animates; hidden exposure highlights	Wow moment
1:35	Bull, Bear, Auditor debate aloud; interrupt the Bear	Agents are genuinely adversarial
2:20	Accept a suggestion; before/after comparison and ranges appear	Actionable and honest
2:40	Flash Regret Replay chart: 2020 prediction versus actual	Credibility
2:55	Close: "Do not find out how risky you are during the crash."	Memorable

Demo safeguards: pre-cache API responses, record a backup video, keep a text input ready, test the microphone and room noise, and avoid live calls to slow external services.

9. Making it internship-ready for fintech firms

Recruiters value rigor and clarity over the number of agents. Aim to be able to defend every number.

- **Resume bullet pattern:** "Built a stress-testing engine calibrated on N historical crises, with adversarial AI agents challenging allocations; reduced simulated maximum drawdown by X% versus a 60/40 baseline." Fill X only with a result you actually measured.
- **Repository quality:** clear README, architecture diagram, one-command setup, tests, CI, sample data, short demo video.
- **Be ready for interview questions:** how you avoided look-ahead bias; why you chose the covariance estimator; what happens when correlations break down; where the LLM is trusted and where it is not; how you handled transaction costs.
- **Tailor the pitch:** risk teams care about VaR/CVaR and stress testing; quant roles about backtest rigor; engineering roles about architecture, reliability and explainability.
- **Market specificity:** NSE/BSE data, RBI rate shocks and rupee-oil sensitivity show thoughtfulness.
- **Write a short limitations section:** linear factor models understate tail events, historical betas may not hold, curated links are assumptions.

10. Risks and mitigations

Risk	Impact	Mitigation
LLM invents impact numbers	Loss of credibility	Quant engine computes all numbers; agents must cite fields; validate outputs
Voice fails during demo	Mandatory feature broken	Text fallback, push-to-talk, recorded backup, pre-tested hardware
API latency or rate limits	Slow, stuttering demo	Cache, parallelize agent calls, limit to one rebuttal round
Overfitting backtests	Misleading results	Walk-forward, out-of-sample crisis tests, report failures
Scope creep	Unfinished core	Freeze scope after Phase 4; cut optional features first
Data licensing or quality	Legal or accuracy issues	Use permitted sources, cache, document limitations
Perceived as financial advice	Compliance concern	Simulation-only banner, disclaimers, no real execution

11. Feature priority if time is short

- **Tier 1 (must ship):** Crisis Twin engine, debate, Voice War Room, propagation map.
- **Tier 2 (credibility):** Regret Replay with calibration chart, uncertainty ranges, audit log.
- **Tier 3 (polish):** Crisis Drill, report card.
- **Tier 4 (cut first):** Why Did My Portfolio Move, behavioral risk profile, extra analytics agents.

Disclaimer: this project is a decision-support and educational simulation. It does not provide financial advice and does not execute real transactions. Verify all hackathon API details (model names, endpoints, limits) in the official documentation.